CSE_2323

Memory and PLD

Digital Logic Design

Course code: CSE-2323
Credit Hour: 3

Md. Mujibur Rahman Maruf

Assistant Lecturer,

Dept. of CSE, IIUC

Mujiburmaruf.cuet17@gmail.com

International Islamic University Chittagong(IIUC)

Primary Memory

What is Primary Memory?

Primary Memory (also called **main memory**) is the memory directly accessible by the CPU. It stores data and instructions that the CPU needs immediately while executing programs. It is typically fast but limited in size and can be either **volatile** (loses data when power is off) or **non-volatile** (retains data).

types of primary memory:

- RAM (Random Access Memory)
- ROM (Read-Only Memory)
- Cache Memory
- Registers

RAM(Random Access Memory)

RAM is a type of **volatile memory** used by computers to store data temporarily while they are powered on. It allows the CPU to read and write data quickly for running programs and processes

- **Types of RAM**

. **Static RAM (SRAM):** Uses flip-flops to store each bit. Faster and more expensive. Does not need to be refreshed.Example: Cache memory inside the CPU.

- **Dynamic RAM (DRAM):** Stores bits as charges in capacitors, which need constant refreshing. Slower but cheaper and higher density. Example: Main memory (system RAM) in most desktop and laptop computers (e.g., DDR4 RAM modules).

Primary Memory

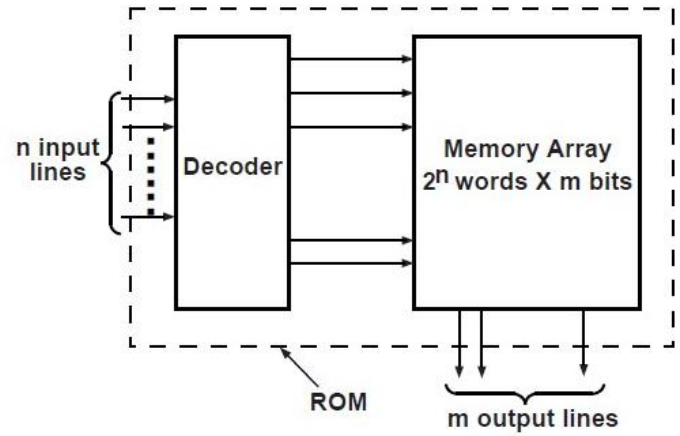
- **Cache Memory**
- Very fast memory located close to the CPU that stores frequently used data to speed up processing
- **Type:** Volatile
- **Example:** L1, L2 cache inside the CPU.

- **Registers**
- Small, very fast storage locations inside the CPU for temporary holding of data and instructions.
- **Type:** Volatile
- **Example:** Accumulator, program counter

ROM

A ROM is a memory (or storage) device in which permanent binary information is stored.

Once the pattern is established, it stays within the unit even when power is turned off and on again



A read-only memory (ROM) is a device that includes both the decoder and the OR gates within a single IC package. The connections between the outputs of the decoder and the inputs of the OR gates can be specified for each particular configuration

Types of ROM

- **Mask ROM (MROM):** Data is permanently written during manufacturing.
- Example: Firmware in a microwave oven or a calculator where the program never changes.
- **Programmable ROM (PROM):** Blank ROM that can be programmed once by the user.
- Example: Early microcontroller boot code PROMs used in embedded systems.
- **Erasable Programmable ROM (EPROM):** Can be erased by UV light and reprogrammed multiple times.
- Example: Intel 2716 EPROM chip used in early computer BIOS.
- **Electrically Erasable Programmable ROM (EEPROM):** Can be erased and reprogrammed electrically without removal.
- Example: EEPROM chips used in modern computers to store BIOS settings.
- **Flash Memory:** A type of EEPROM with block-wise erase and fast write.
- Example: USB flash drives, SSD drives, and memory cards.

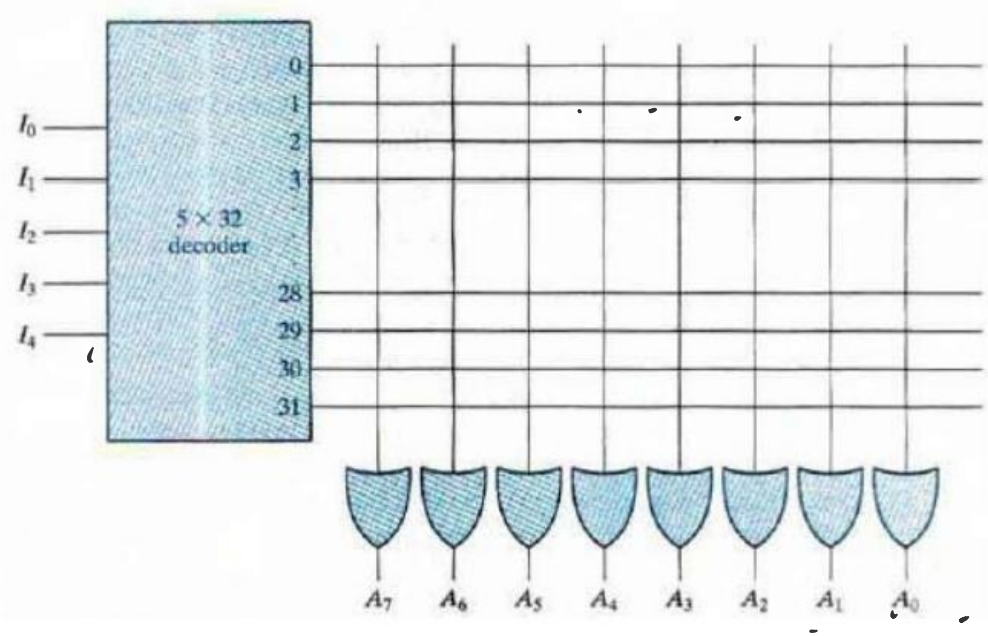
5-to-32 ROM (8-bit Data per Line)

Addressing:
- Uses 5 address lines → 2^5 = 32 unique addresses
- Each address accesses one memory row

Decoder:
- A 5-to-32 decoder activates one of 32 output lines
- Each line selects a memory row

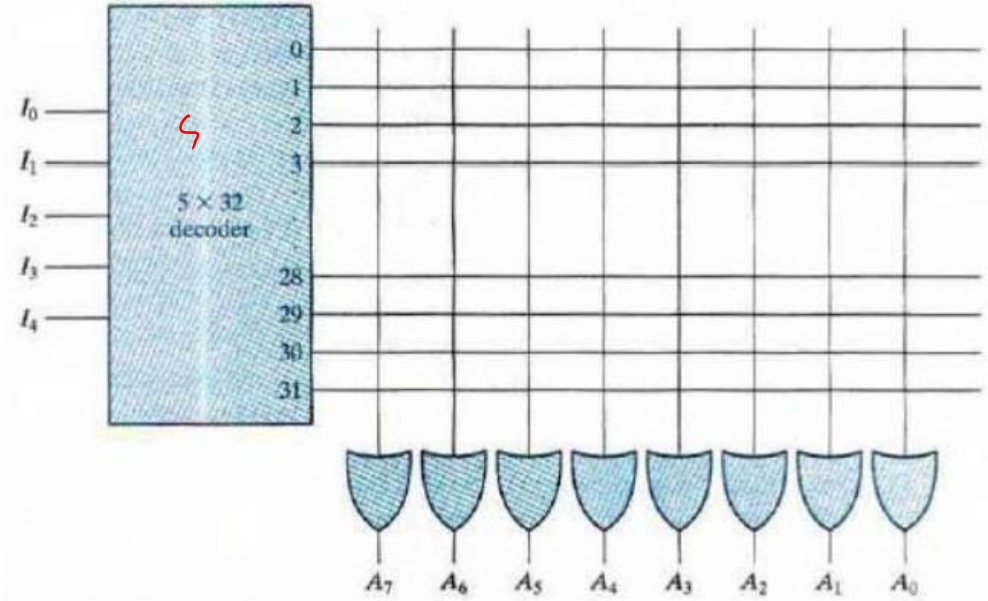
Memory Matrix:
- Each row stores 8 bits (e.g., 10101011)
- Bits are hardwired as 1 (connected) or 0 (unconnected)

Output Generation:
- 8 columns (bit lines) are connected to 8 OR gates
- Only the selected row's bits affect the OR gates
- OR gates output the 8-bit stored data



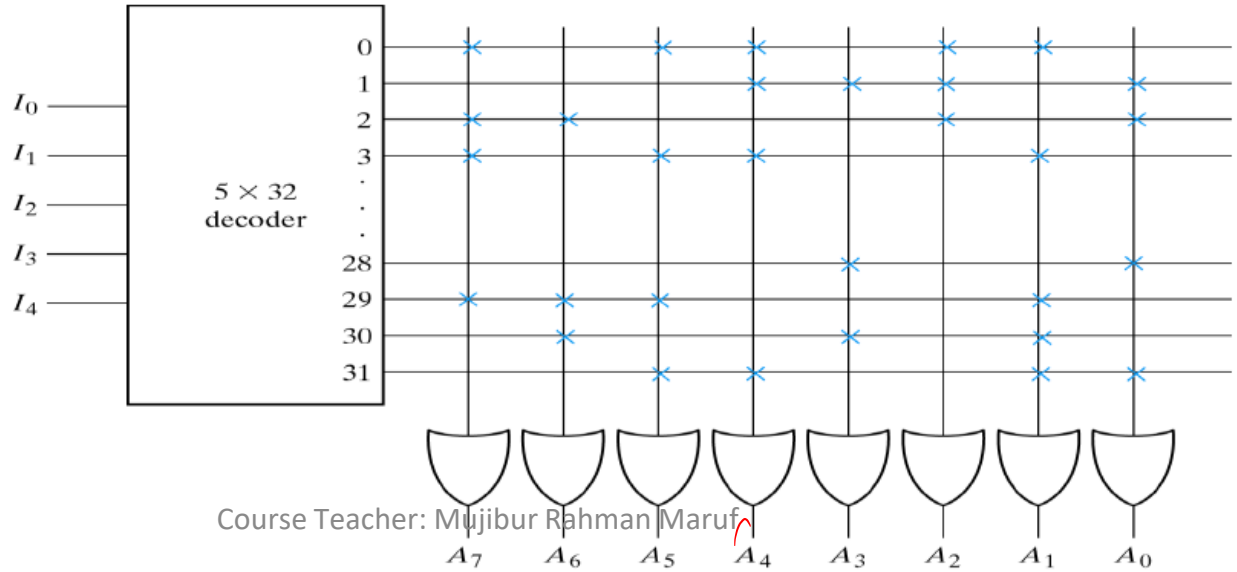
◇ **Example:**
- Input Address: 00101 (decimal 5)
- Row 5 activated, suppose data = 11001010
- OR gates output: 11001010

5-to-32 ROM (8-bit Data per Line):example



ROM Truth Table (Partial)

Inputs					Outputs							
I_4	I_3	I_2	I_1	I_0	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
0	0	0	0	0	1	0	1	1	0	1	1	0
0	0	0	0	1	0	0	0	1	1	1	0	1
0	0	0	1	0	1	1	0	0	0	1	0	1
0	0	0	1	1	1	0	1	1	0	0	1	0
⋮								⋮				
1	1	1	0	0	0	0	0	0	1	0	0	1
1	1	1	0	1	1	1	1	0	0	0	1	0
1	1	1	1	0	0	1	0	0	1	0	1	0
1	1	1	1	1	0	0	1	1	0	0	1	1



Example

This ROM uses a **3-to-8 decoder** and **OR gates** to implement functions f,g,h,

Inputs a,b,c generate address lines 0-7 through the decoder.

At the intersection of each decoder output and function line, a **cross (×)** shows a hardwired logic 1.

OR gates sum the active lines:

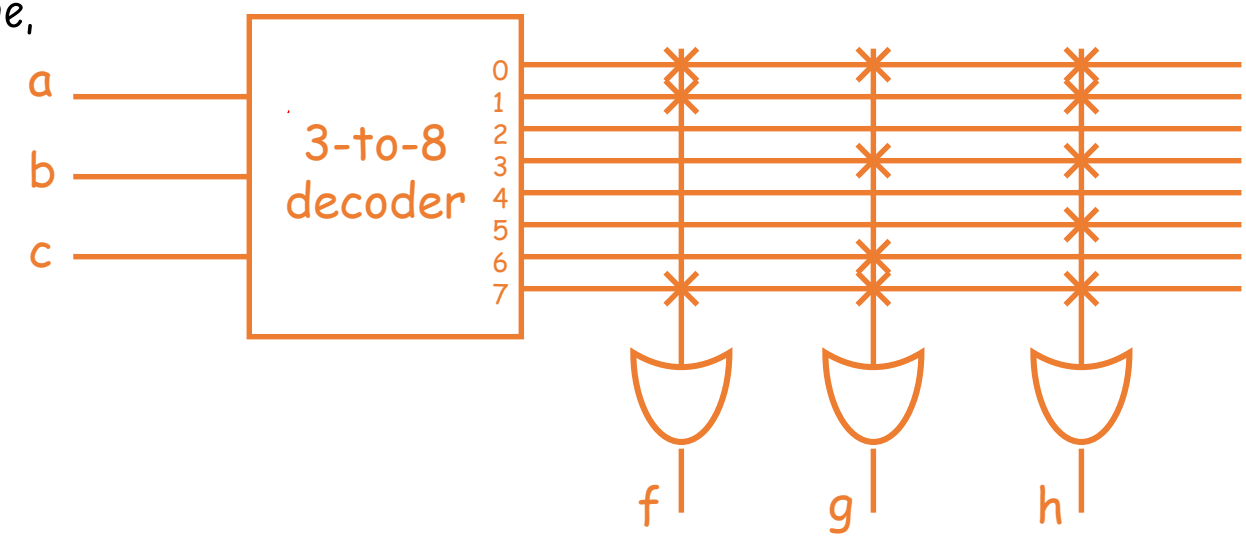
- $f=\sum m(0,1,7)$ → crosses at 0,1,7
- $g=\sum m(0,3,6,7)$ → crosses at 0,3,6,7
- $h=\sum m(0,1,3,5,7)$ → crosses at 0,1,3,5,7

Implement the function with ROM.

$f = \sum m(0, 1, 7)$

$g = \sum m(0, 3, 6, 7)$

$h = \sum m(0, 1, 3, 5, 7)$



Implement the following function with ROM $F(w,x,y,z)=\sum(0,1,3,4,8,9,15)$ Self task

PLD (Programmable Logic Device)

- A PLD is a digital IC that can be **programmed** to perform various logic functions. It provides flexibility in circuit design.
- **General Structure:**
- **Inputs** - Receive external signals.
- **AND Array** - Generates programmable product terms.
- **OR Array** - Combines product terms into logic functions.

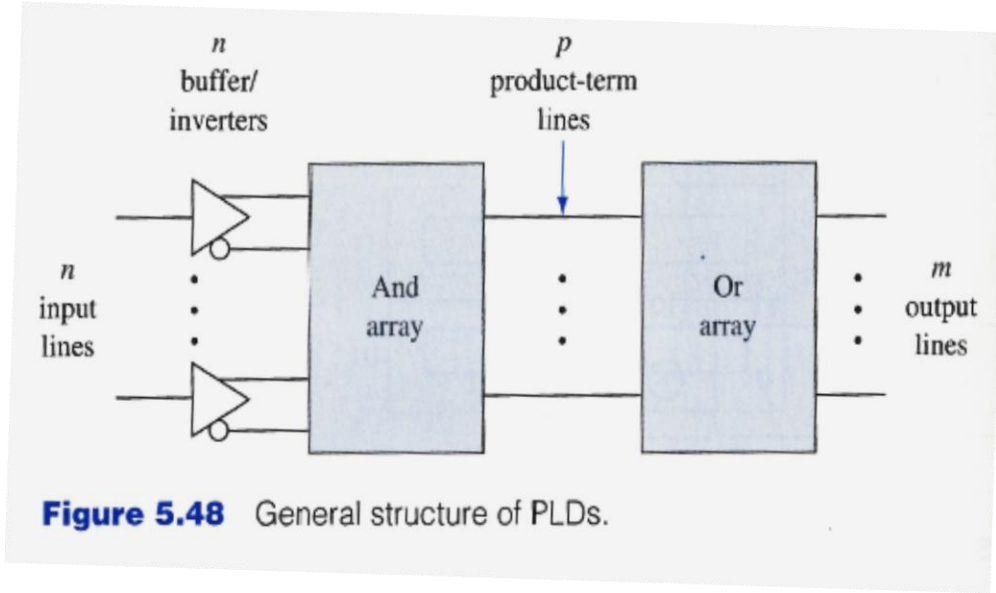


Figure 5.48 General structure of PLDs.

General Structure of PLD

Classification of PLD

Device	AND-array	OR-array
PROM	Fixed	Programmable
PLA	Programmable	Programmable
PAL	Programmable	Fixed

Programmable Logic Array(PLA)

A **PLA** is a type of PLD that allows **programmable AND and OR arrays** to implement **combinational logic functions**.

Key Features:

Programmable AND array: Creates custom product terms.

Programmable OR array: Combines product terms to form desired outputs.

More flexible than PAL but slower and costlier.

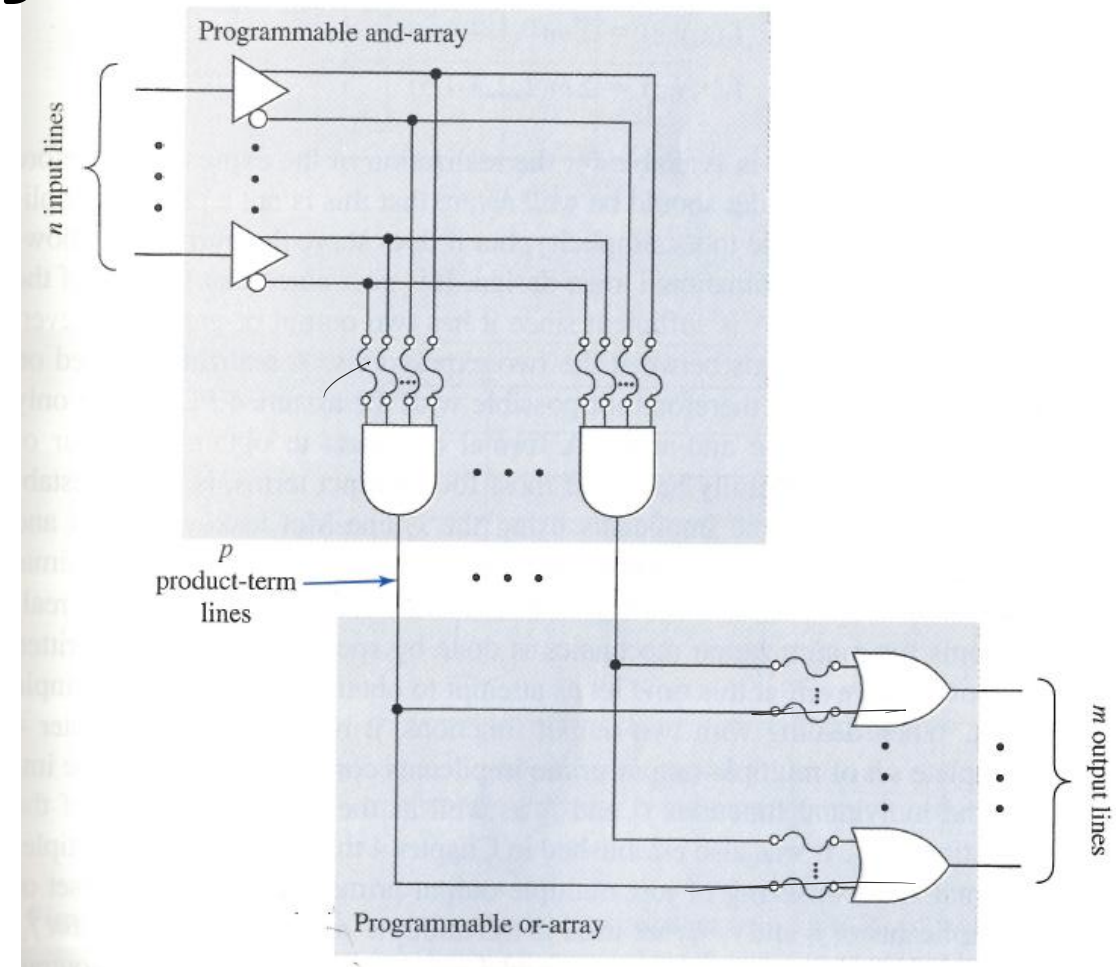


Figure 5.55 Logic diagram of an $n \times p \times m$ PLA.

Programmable Array Logic(PAL)

A **PAL** is a type of PLD where the **AND array is programmable**, but the **OR array is fixed**. It is mainly used to implement **combinational logic** efficiently.

Key Features:

Programmable AND array: User can define custom product terms.

Fixed OR array: Each output has a predefined OR gate.

Faster and simpler than PLA, but less flexible.

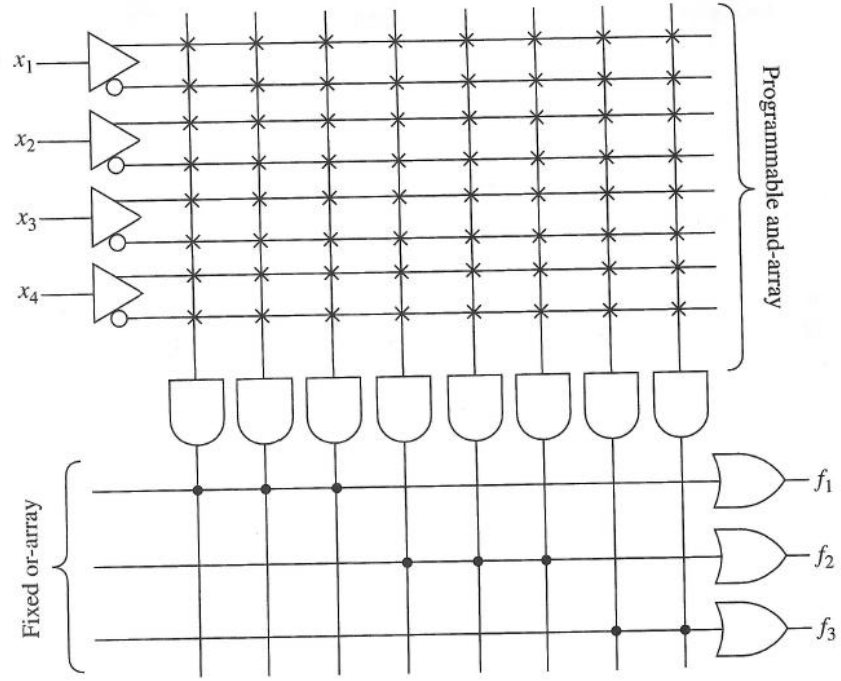


Figure 5.62 A simple four-input, three-output PAL device.

examples:

4-input, 3-output PAL device

Three Boolean expressions can be realized in which two expressions can have at most 3 product terms and one expression can have at most 2 product terms.

Example:

A.Implement Boolean functions using PLA

B.Implement Boolean functions using PAL

Class work

$f = \sum m(0, 1, 7)$
$g = \sum m(0, 3, 6, 7)$
$h = \sum m(0, 1, 3, 5, 7)$

Step:1 . Simplify functions Using K-Map:
Step 2: List Product Terms (AND Terms):
Step 3:Program Arrays:
For PLA:

Program the **AND array** for all required product terms.
Program the **OR array** to combine needed terms per output.

For PAL:

Program the **AND array** for each output function's terms.
Use the **fixed OR array** (per output) to combine product terms.

Example:

A combinational circuit is defined by the functions:

Class work

$F1(A, B, C) = \Sigma(3, 5, 6, 7)$

$F2(A, B, C) = \Sigma(0, 2, 4, 7)$

Implement the circuit with a PLA having three inputs, four product terms, and two outputs.